Processes and threads

A process is a program in execution. Each process has its own address space, a program counter, a set of registers, and open files. The operating system keeps this information in a process control block (PCB). When the CPU switches from one process to another, the OS saves the state of the old process into its PCB and loads the state of the new one. This is called a context switch.

A thread is a lightweight unit of execution inside a process. Threads in the same process share the address space, global variables, and open files, but each thread has its own program counter, registers, and stack. Creating a thread is cheaper than creating a process because no new address space is needed.

Process states

Process states: new, ready, running, waiting, and terminated. A process in the ready state is waiting only for the CPU. A process in the waiting state is blocked on some event, such as disk I/O finishing. Only one process per CPU core can be in the running state at a time.

CPU scheduling: FCFS and SJF

CPU scheduling decides which ready process gets the CPU next. The scheduler aims to maximise CPU utilisation and throughput while minimising turnaround time, waiting time, and response time. Preemptive schedulers can take the CPU away from a running process; non-preemptive schedulers wait for the process to block or finish.

First-Come First-Served (FCFS) runs processes in arrival order. It is simple but suffers from the convoy effect: one long CPU-bound process makes every short process behind it wait. Average waiting time can be very high.

Shortest Job First (SJF) runs the process with the smallest next CPU burst. SJF gives the minimum possible average waiting time, but the OS cannot know burst lengths in advance, so real systems estimate them with an exponential average of past bursts. The preemptive version is called Shortest Remaining Time First (SRTF).

Round Robin and priority scheduling

Round Robin (RR) gives each process a fixed time quantum, typically 10 to 100 milliseconds, then moves it to the back of the ready queue. A very small quantum causes too many context switches; a very large quantum turns RR into FCFS. A common rule of thumb is that 80 percent of CPU bursts should be shorter than the quantum.

Priority scheduling runs the highest-priority ready process. Its main problem is starvation: a low-priority process may never run. The fix is aging, which gradually raises the priority of processes that have waited a long time.

Deadlock

A deadlock is a situation where a set of processes are each waiting for a resource held by another process in the set, so none of them can ever proceed. Deadlock requires four conditions to hold at the same time: mutual exclusion, hold and wait, no preemption, and circular wait. These are the Coffman conditions.

Deadlock handling strategies: prevention (break one of the four conditions in advance), avoidance (the Banker's algorithm checks that every allocation leaves the system in a safe state), detection and recovery (let deadlocks happen, find them with a wait-for graph, then kill or roll back a process), and ignoring the problem, which is what most general-purpose operating systems actually do.

Banker's algorithm, races, and locks

The Banker's algorithm requires each process to declare its maximum resource need up front. A request is granted only if the resulting state is safe, meaning there is some order in which every process could finish. It is rarely used in practice because the maximum needs are usually unknown.

A race condition occurs when two threads access shared data at the same time and the result depends on the order in which they run. The section of code that touches the shared data is called a critical section. Mutual exclusion guarantees that only one thread is in its critical section at a time.

A mutex is a lock with two operations: acquire and release. A semaphore is a counter with wait (P) and signal (V) operations. A binary semaphore behaves like a mutex; a counting semaphore allows up to N threads into a resource pool at once.

Amdahl's law

Amdahl's law estimates the speedup from adding cores. If a fraction S of a program is serial, the maximum speedup with N cores is 1 / (S + (1 - S) / N). For example, if 10 percent of a program is serial, 8 cores give at most about 4.7 times speedup, not 8 times.